Analyzing Diamond Prices and Shapes 💎

Diamonds are one of the most popular and valuable gemstones in the world. However, their prices can vary greatly based on a number of factors, including the diamond's shape, weight, clarity, colour, cut, polish, symmetry, fluorescence and measurements. In this notebook, we will explore how diamond prices vary across different shapes using the Natural Diamonds Prices & Images dataset from Kaggle.

The dataset contains information on over 6,000 diamonds, including their shape, weight, clarity, colour, cut, polish, symmetry, fluorescence and measurements with their price. We will use pandas, a popular data manipulation library in Python, to analyze and visualize the data. Our goal is to gain insights into the factors that influence diamond prices and how these factors vary across different shapes.

To begin, we will use the opendatasets library to download the dataset from Kaggle. We will then read in the data for each diamond shape and concatenate it into a single dataframe. We will print information about the dataframe using `df.info()`, which will give us insights into the data types and missing values in the dataset.

We will save the concatenated dataframe to a CSV file for future use. Then, we will explore the data using various pandas functions and visualizations. Finally, we will draw conclusions based on our analysis and discuss the insights we have gained.

Importing required libraries 📦

We are using opendatasets and pandas libraries for this notebook

In [ ]:
```
import opendatasets as od
import pandas as pd
```

Data Collection 🔎

First, we'll use the opendatasets library to download the dataset from Kaggle:

In [ ]:
```
dataset_url = (
    "https://www.kaggle.com/datasets/harshitlakhani/natural-diamonds-prices-images"
)
od.download(dataset_url)
```

```
Skipping, found downloaded files in "./natural-diamonds-prices-images" (use force=True to force download)
```

This will download the dataset and save it to a directory called natural-diamonds-prices-images.

Data Preparation 🔧

Next, we'll read in the data for each diamond shape and concatenate it into a single dataframe:

In [ ]:
```
data_cushion = pd.read_csv(
    "./natural-diamonds-prices-images/Diamonds2/data_cushion.csv"
)
data_emerald = pd.read_csv(
    "./natural-diamonds-prices-images/Diamonds2/data_emerald.csv"
)
data_heart = pd.read_csv("./natural-diamonds-prices-images/Diamonds2/data_heart.csv")
data_marquise = pd.read_csv(
    "./natural-diamonds-prices-images/Diamonds2/data_marquise.csv"
)
data_oval = pd.read_csv("./natural-diamonds-prices-images/Diamonds2/data_oval.csv")
data_pear = pd.read_csv("./natural-diamonds-prices-images/Diamonds2/data_pear.csv")
data_princess = pd.read_csv(
    "./natural-diamonds-prices-images/Diamonds2/data_princess.csv"
)
data_round = pd.read_csv("./natural-diamonds-prices-images/Diamonds2/data_round.csv")
```

Concatenating data for all diamond shapes into a single dataframe & displaying the concatenated dataframe

In [ ]:
```
df = pd.concat(
    [
        data_cushion,
        data_emerald,
        data_heart,
        data_marquise,
        data_oval,
        data_pear,
        data_princess,
        data_round,
    ],
    ignore_index=True,
)
df
```

Out[ ]:

	Id	Shape	Weight	Clarity	Colour	Cut	Polish	Symmetry	Fluorescence	Messurements	Price
0	1638147	CUSHION	0.55	SI2	E	EX	EX	VG	N	5.05-4.35×2.94	1,378.65
1	1630155	CUSHION	0.50	VVS1	FANCY	EX	EX	VG	F	4.60-4.31×2.92	1,379.74
2	1612606	CUSHION	0.51	VS2	H	EX	EX	VG	N	4.71-4.35×2.94	1,380.19
3	1638140	CUSHION	0.50	VS2	H	EX	EX	VG	N	4.91-4.26×2.88	1,380.61
4	1536093	CUSHION	0.53	SI1	D	EX	VG	VG	N	4.70-4.46×3.01	1,383.13
...	...	...	...	...	...	...	...	...	...	...	...
6334	1751206	ROUND	0.50	SI1	E	EX	EX	EX	N	5.00-5.04×3.18	2,453.10
6335	1773024	ROUND	0.50	SI1	E	EX	EX	EX	N	5.03-5.06×3.16	2,453.10
6336	1789435	ROUND	0.50	VS2	D	VG	EX	VG	N	4.93-4.96×3.19	2,453.20
6337	1774887	ROUND	0.52	SI1	H	EX	EX	EX	N	5.11-5.13×3.20	2,453.41
6338	1630512	ROUND	0.50	VS1	H	EX	EX	VG	N	5.09-5.13×3.12	2,453.69

6339 rows × 11 columns

Printing information about the dataframe ℹ️

Using `df.info()` we will get info about the dataset

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6339 entries, 0 to 6338
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Id            6339 non-null   object
 1   Shape         6339 non-null   object
 2   Weight        6339 non-null   float64
 3   Clarity       6319 non-null   object
 4   Colour        6339 non-null   object
 5   Cut           6337 non-null   object
 6   Polish        6338 non-null   object
 7   Symmetry      6332 non-null   object
 8   Fluorescence  6337 non-null   object
 9   Messurements  6339 non-null   object
 10  Price         6339 non-null   object
dtypes: float64(1), object(10)
memory usage: 544.9+ KB
```

Insights form the dataset 💡

- The DataFrame has a total of 6339 entries (rows).
- There are 11 columns in the DataFrame.
- The columns in the DataFrame and their data types are:

  ```
  Id: object (string)
  Shape: object (string)
  Weight: float64 (floating point number)
  Clarity: object (string)
  Colour: object (string)
  Cut: object (string)
  Polish: object (string)
  Symmetry: object (string)
  Fluorescence: object (string)
  Messurements: object (string)
  Price: object (string)
  ```

- The Id, Shape, Weight, Colour, Messurements, and Price columns have no missing values (i.e., non-null count is equal to the total number of entries).
- The Clarity, Cut, Polish, Symmetry, and Fluorescence columns have some missing values (i.e., non-null count is less than the total number of entries).
- The memory usage of the DataFrame is 544.9 KB.

Saving the concatenated dataframe to a CSV file 📎

Save the dataframe in `./../data/diamonds.csv` without index

In [ ]:
```
df.to_csv("./../data/diamonds.csv", index=False)
```

Conclusion 🎉

In this notebook, we explored the Natural Diamonds Prices & Images dataset from Kaggle to analyze how diamond prices vary across different shapes. We used the opendatasets and pandas libraries to download, read and concatenate the data for each diamond shape into a single dataframe. Then, we printed information about the dataframe using df.info(), which showed that the DataFrame has a total of 6339 entries (rows), with 11 columns in the DataFrame. We also observed that some columns such as Clarity, Cut, Polish, Symmetry, and Fluorescence have some missing values. Finally, we saved the concatenated dataframe to a CSV file without an index. This analysis provides an overview of the dataset and can be used as a starting point for further exploration and analysis.

Reference 📄

Natural Diamonds Prices & Images dataset from Kaggle: Natural Diamonds Prices & Images dataset